

COURSE: Software Design Patterns (ShP-2216)

Assemgazy Olzhas – SE-2528

Assignment 2: Factory Method and Abstract Factory

This report describes a Java logistics application that supports road and sea delivery with Windows and macOS user interface components. Factory Method is used to create the transport objects, and Abstract Factory is used to create matching pairs of UI components for each platform.

GitHub repository: https://github.com/iuve1n/SDP_Assignment2

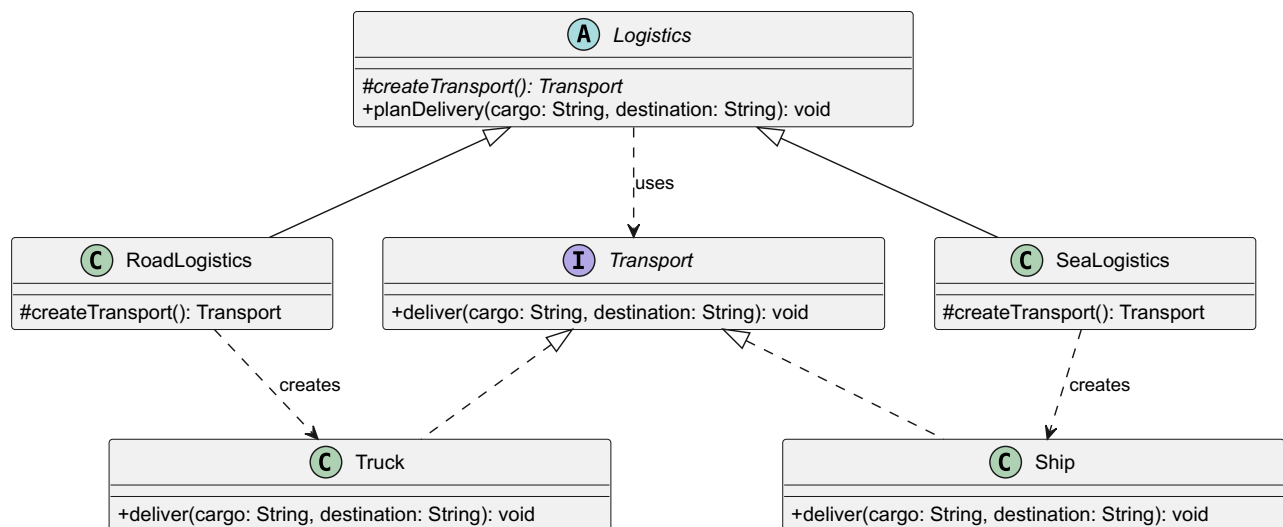
1. Introduction

The application asks the user to choose a delivery mode (ROAD or SEA) and a UI platform (WINDOWS or MACOS). Based on these two choices, it selects a concrete Logistics object and a concrete GUIFactory object, then runs the delivery and rendering logic through their shared interfaces. Factory Method fits the transport part of the task because each delivery mode needs its own object creation step, but the delivery workflow itself stays the same. Abstract Factory fits the UI part because a button and a checkbox must always come from the same platform family, and this pattern guarantees that they do.

2. Part A – Factory Method

Factory Method is used to create the transport objects. Each delivery mode builds its own transport, while the delivery steps themselves are written only once and shared by both modes.

Diagram 1 – Factory Method class diagram

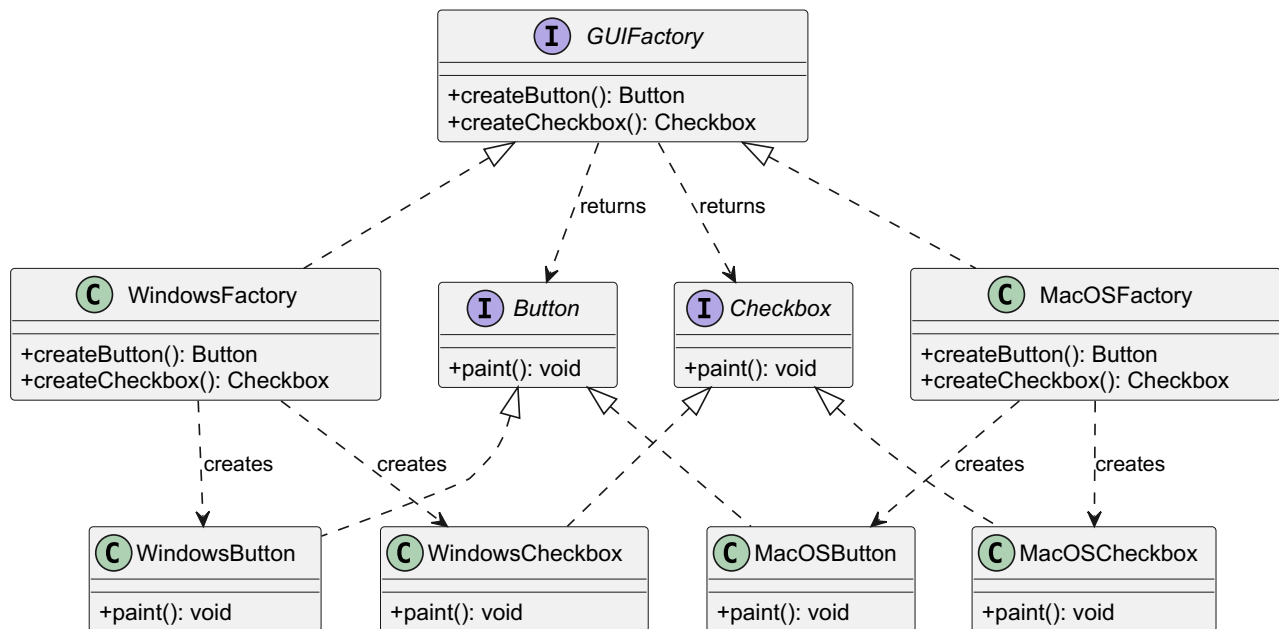


Transport is the product interface with one method, `delivery(cargo, destination)`. Truck and Ship are the concrete products, and each prints a different delivery message. Logistics is an abstract creator class that declares `createTransport()` and also holds the shared `planDelivery(...)` method. RoadLogistics and SeaLogistics are the concrete creators: each overrides `createTransport()` and returns its own transport object. `planDelivery(...)` never checks which subclass it is running in – it only calls `createTransport()` and then calls `delivery()` on the result. Because the client depends only on the abstract Logistics type, a new delivery mode can be added later without touching the existing code.

3. Part B – Abstract Factory

Abstract Factory is used to create a matching pair of UI components for each platform, so a Windows button never ends up next to a macOS checkbox.

Diagram 2 – Abstract Factory class diagram



Both concrete factories implement the same GUIFactory interface, so the client can be given either one without any change to its own code.

Button and Checkbox are the abstract product interfaces, each with a paint() method. WindowsButton, WindowsCheckbox, MacOSButton and MacOSCheckbox are the concrete products. GUIFactory declares createButton() and createCheckbox(), and WindowsFactory and MacOSFactory implement it, each returning components from its own family. DeliveryApplication receives a GUIFactory and a Logistics object through its constructor. It calls createButton() and createCheckbox() on the factory and renders both through their interfaces, so the concrete platform classes never appear in the client code.

4. Clean Code Evidence

Five Clean Code practices are demonstrated below, using short excerpts from the submitted project. The last two are connected to Clean Code, Chapter 6 (Objects and Data Structures).

4.1 Meaningful names

```
public abstract class Logistics {    public abstract Transport
createTransport();    public void planDelivery(String cargo, String
destination) { ... } }
```

Names such as Logistics, createTransport and planDelivery describe the domain and the pattern role at the same time, so the class does not need extra comments to explain its purpose.

4.2 Small methods, single responsibility

```
private static Logistics selectLogistics(String mode) {
if (mode.equals("ROAD")) return new RoadLogistics();
if (mode.equals("SEA")) return new SeaLogistics();
return null; }
```

Input validation, object selection, rendering and delivery are each handled by a separate short method, so every method is easy to read and to test on its own.

4.3 Avoiding duplicated logic

```
public void planDelivery(String cargo, String destination) {  
    Transport transport = createTransport();  
    transport.delivery(cargo, destination); }  
}
```

This workflow is written once in Logistics and reused by both RoadLogistics and SeaLogistics, so the delivery steps are never copied between the two subclasses.

4.4 Data abstraction (Chapter 6)

```
Transport transport = logistics.createTransport();  
transport.delivery(cargo, destination);
```

The client only talks to the Transport interface. It cannot see whether a Truck or a Ship was created, so a new transport type can be added later without changing this code.

4.5 Encapsulation (Chapter 6)

```
public class DeliveryApplication {  
    private final GUIFactory factory;  
    private final Logistics logistics;  
    DeliveryApplication(GUIFactory factory, Logistics logistics) {  
        this.factory = factory;  
        this.logistics = logistics;  
    }  
}
```

The factory and logistics fields are private and set once through the constructor. There are no public setters, so the object's dependencies cannot be changed from outside after it is built.

5. Verification

All six required checks were run on the submitted project. Results are summarised below; full console transcripts are included in the repository.

#	Input	Expected result	Result
1	ROAD + WINDOWS	Truck delivery; Windows button and checkb	x Pass
2	SEA + WINDOWS	Ship delivery; Windows button and checkbox	Pass
3	ROAD + MACOS	Truck delivery; macOS button and checkbox	Pass
4	SEA + MACOS	Ship delivery; macOS button and checkbox	Pass
5	Invalid mode, valid platform	Validation message; no delivery	Pass
6	Valid mode, invalid platform	Validation message; no UI built	Pass

Missing input: if the console input is empty or unreadable, the application prints a message asking for the value again and does not fall back to a default delivery mode or platform.

6. Comparison and Design Reflection

6.1 Simple Factory versus Factory Method

A Simple Factory hides object creation behind one method with an if or switch statement. Factory Method moves that decision into subclasses through an overridden method. The project uses Factory Method because Logistics needed a shared planDelivery(...) workflow that both delivery modes reuse – a Simple Factory would not provide subclasses to attach that shared behaviour to. Adding a delivery mode this way only means adding one subclass, not editing a switch statement.

6.2 Factory Method versus Abstract Factory

Factory Method produces one kind of product (a Transport). Abstract Factory produces a matching set of several products from the same family (a Button and a Checkbox that belong to the same platform). The two patterns are combined in this project: Factory Method picks the delivery behaviour, and Abstract Factory keeps the UI pair consistent, and the two choices do not interfere with each other.

6.3 Extending the application

New transport (for example Air): add an Airplane class implementing Transport and an AirLogistics class extending Logistics. Update the startup selection to accept "AIR". Transport, planDelivery(...) and the existing subclasses stay unchanged.

New UI family (for example Linux): add LinuxButton and LinuxCheckbox implementing the existing interfaces, and a LinuxFactory implementing GUIFactory. Update the startup selection to accept "LINUX". DeliveryApplication and the other factories stay unchanged.

New UI product (for example a text field): add a TextField interface and add createTextField() to GUIFactory. Every concrete factory must then implement it, and DeliveryApplication must call and render it. This change touches every factory, unlike the two cases above.

7. Conclusion

Factory Method and Abstract Factory solve two different problems in this project. Factory Method lets each delivery mode create its own transport while reusing one shared delivery workflow. Abstract Factory keeps a UI pair consistent by grouping related products into one family per platform. Used together, both choices can change at startup without any conditional logic inside the delivery or rendering code itself.

References

- Freeman, E. & Robson, E. (2004). Head First Design Patterns. O'Reilly Media. Chapter 4.
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. Chapter 6.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns. Addison-Wesley.
- Software Design Patterns syllabus 2026-2027, Astana IT University.